

Automatic pear and apple detection by videos using deep learning and a Kalman filter

KENTA ITAKURA,^{1,2} YUMA NARITA,¹ SHUHEI NOAKI,³ AND FUMIKI HOSOI^{1,*}

¹Graduate school of Agricultural and Life Sciences, The University of Tokyo, Yayoi 1-1-1, Bunkyo-ku, Tokyo, 113-8657, Japan

²JSPS researcher (DC1), Japan

³CULTA Inc., 4-20-3, Ebisu, Shibuya-ku, Tokyo, 150-6027, Japan

*ahosoi@mail.ecc.u-tokyo.ac.jp

Abstract: Pears and apples in videos recorded while walking were detected automatically using a deep-learning-based method referred to as YOLO. The same fruits in the successive video frames were then identified using a Kalman filter. The average precision of the pear detection was 0.97, while the number of correctly counted pears was 226, out of 234. A YOLO v2 network with a larger input image size and data augmentation method contributed to the high accuracy in the counting. The pears and apples in the videos were counted automatically, within an absolute error of 10% under unstable light conditions and with greenish fruits.

© 2021 Optical Society of America under the terms of the [OSA Open Access Publishing Agreement](#)

1. Introduction

A precise estimation of the number of fruits in an orchard is important. It can provide yield and maturity estimations, planning of recreations such as fruit picking, evaluation of the damage by a disaster, and recognition system for robots in agricultural automation.

The fruit counting can be divided into two essential steps, fruit detection and fruit tracking using a sequence of image frames, which capture the target trees. To record the target areas in the orchard, numerous images should be acquired so that video recording is required. To count the fruits in the video, the correspondence of each fruit in the adjacent frames is essential. A certain fruit captured in frames should be identified and other fruits should be simultaneously differentiated. However, tracking the same fruits for identification is challenging because the location and appearance vary depending on the frames owing to ambient factors such as illumination conditions. Yamamoto et al. (2014) [1] detected tomatoes in a still image by combining rough pixel classification, blob segmentation, and X-means clustering. Takemoto et al. (2019) [2] proposed a method for detecting plums using a deep learning-based technique. These methods have been used to count fruits using a still image and handle a larger area. Fruit counting using a video is needed. Song et al. (2014) [3] presented a method to calculate the number using multiple images. Santos et al. (2020) [4] detected grapes using a video by a deep learning method, Mask R-CNN (He et al., 2017) [5], and used a photogrammetry approach referred to as structure from motion to reconstruct three-dimensional information of the orchard, which enabled fruit counting. An improved deep neural network, DaSNet-v2, which can be used for detection and instance segmentation on fruits, and semantic segmentation on branches were demonstrated for apple detection [6]. Wan and Goudos created a multi-labeled and knowledge-based outdoor orchard image library using 4000 images in the real world [7].

As stated above, numerous studies have been carried out on fruit detection. However, a few methods for detection and tracking of fruits for counting, particularly with greenish fruits and unstable light conditions, such as under the canopy, have been reported. Even if a certain fruit is hidden by leaves or branches and the detection is failed, tracking the fruit is necessary; otherwise, the fruit will be double-counted, once the fruit is detected again in the successive video frames.

In this study, we recorded a video of pears and apples, including green apples, to automatically calculate their numbers. The fruits were detected in each frame using a deep learning-based object detection algorithm. Each detected fruit was then tracked for the correspondence of the fruits in the successive video frame under the unstable light conditions beneath the tree canopies.

2. Materials and methods

2.1. Data acquisition

The data for pear and apple detection were acquired in different orchards. The video of the pears was recorded in Hatsuseien, Matsudo city, Chiba prefecture, Japan on 20 August. The cultivars were Kosui, Hosui, Kaori, and Niitaka. A video of apples was acquired in Nagano Fruits Production in Nagano city, Nagano prefecture, Japan on 23 September. The cultivars were Fuji, Shinano Sweet, and Akibae. The videos were recorded while walking using an iPhoneX (Apple Inc., USA). Figure 1 shows two frames capturing the pears. The circled pears in panels [a] and [b] are identical. Thus, the pears should be tracked after the detection. The heights of the pear trees were approximately 1.5 m. The fruits were under the canopy. The video was then recorded under the tree crowns. The resolution and frame rate of the video were 1920×1080 pixels and 30 fps, respectively. After the image acquisition, the object detector was made to count the number of fruits as mentioned below. The workflow in this study is shown in Fig. 2.



Fig. 1. An example of pears in video frames. The circled pears in the panel [a] and [b] are identical, which should be tracked in successive video frames for the counting.

2.2. Pear and apple detection

An object detection algorithm using deep learning, YOLO v2 [8], was used for fruit detection. For comparison, other convolutional neural networks for object detection, such as YOLO v3 [9] and single-shot detector [10], were also used. The pears and apples in the videos were manually labelled for training. The fruits in the video frames were detected manually and rectangles were assigned around each fruit. The total numbers of labelled pears and apples were 6341 and 7494 captured in 1077 and 526 images, respectively. The software used for the annotation was MATLAB 2020b (MathWorks, USA). The Video Labeler app in MATLAB was utilized. The object detection and tracking were also performed with MATLAB. This number includes the same fruits in different frames. As mentioned below, these frames were randomly divided into training and test datasets. Our fruit counting system was evaluated using the fruit counting result, not the detection accuracy with the fruits in the test image frames. The pear and apple data were analysed separately. However, the method for the counting of pear and apple is almost same. In each dataset, the images were divided into training, validation, and test datasets (8:1:1) to train and test the detection network. The validation data were used for the parameter settings

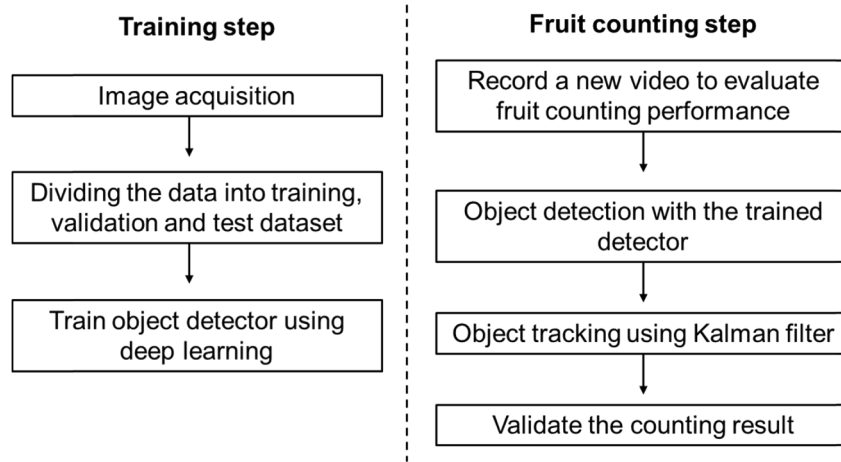


Fig. 2. Flowchart of this study.

of the detection network. The network was trained with the training dataset. The detection performance was evaluated using validation data. After this parameter optimisation, detection was performed with the test dataset. Data augmentation was performed during the training for image variety. The input image was flipped horizontally. The colour information was altered randomly. For the alternation of the colour information, the red–green–blue image was converted into hue–saturation–value. The hue, saturation, and value were randomly changed by -20% to $+20\%$. Data augmentation is frequently performed for image classification tasks using deep learning where the training images are randomly, for example, flipped and rotated. This type of data augmentation was implemented in this study. As our task is object detection, not image classification, the location of bounding boxes to identify the fruits was corrected based on the augmentation. For example, the input image was rotated by 30° and the position of the bounding boxes was rotated also by 30° . The reflection of images horizontally was done with 50% probability. The vertical reflection was not performed.

The size of the input images was set to 608×608 pixels. A pretrained YOLO v2 with an input size of 608×608 pixels presented by Redmon and Farhadi (2017) [8] was utilised. This input size affects the detection accuracy. For comparison, a smaller image input size was tested. A YOLO v2 network with a backbone network of ResNet-18 [11] was built and the detection accuracy with the test dataset was calculated. The parameters for learning such as the initial learning rate, optimiser, and number of epochs were determined manually at 120, while the accuracy was evaluated using the validation dataset. Stochastic gradient descent (SGD) and adaptive moment estimation (ADAM) optimisers were used [12]. The parameters for training the detector follow. The initial learn rate was 1×10^{-2} . The momentum and L2 regularization were 0.9 and 1×10^{-4} , respectively. The minibatch size was 4. The used personal computer had an Intel Core i7-9700F central processing unit, random-access memory of 16 GB, and GeForce RTX 2070 graphics processing unit (NVIDIA Corporation, USA). The evaluation metric for object detection was the average precision [13].

2.3. Object tracking for counting the fruits using a Kalman filter after their detection

For counting the fruits in the video, tracking the identical fruits is essential. As stated above, the circled pears in Fig. 1(a) and (b) are identical and the counting cannot be carried out without the identification. The Kalman filter was applied to the sequence of the video frame [14]. The Kalman filter predicts the object's future location. This function enables to track the detected

fruits even when the object failed to be detected during the tracking. The pear is recorded under the canopy, so that the illumination condition is very changeable. Also, a pear might be occluded by leaves while it is tracked. The Kalman filter was selected for the object tracking considering the potential failure of the detection. A discrete-time linear Kalman filter was used to track the positions and velocities of the detected objects. The Kalman filter was used as an estimator to predict and correct the state of the tracked fruits. In the Kalman filter, we considered a tracking system where X_k is the state vector, which represents the dynamic behaviour of the object, where the subscript k indicates the discrete time. The objective was to estimate X_k by measurements [15]. For the Kalman filter, a parameter called deletion threshold was set at [5,5]. It means, if a confirmed track is not assigned to any detection 5 times in the last 5 tracker updates, then the track is deleted. If the fruit is occluded for five frames in a row, the fruit information is removed. To validate the effectiveness of the Kalman filter in fruit counting, the Kanade–Lucas–Tomasi (KLT) feature-tracking algorithm [16] was used for the fruit tracking. Image features in each bounding box were extracted using the minimum-eigenvalue algorithm [17]. For the evaluation of the fruit counting, a video for neither the training nor test data for the YOLO detector was used. The pears were counted manually and the result of the automated counting was compared.

3. Result

Figure 3(a) shows the pear detection results. The pears were successfully detected as indicated by the bounding boxes. The training lasted approximately 9 h. The average precision for the evaluation of the pear detection was 0.97. The average precision without the data augmentation was 0.88. The average precisions with different methods are listed in Table 1. Figure 4 shows an enlarged view of the video frame after the pear detection. As indicated by the red circle, the pear partly covered by a leaf and overlapped pear could be detected. The average precision for the apple detection was 0.97, which implies that the apple detection was also carried out accurately.



Fig. 3. Pear and apple detection and counting result. The panel [a] shows the pear detection as the rectangle shows. In the panel [b], each pear was differentiated by the different color of rectangle. The object tracking with Kalman filter enabled this counting. The panel [c] shows the counting result with apple.

Table 1. Average precision in pear detection with different methods

Method	Average precision
YOLO v2 with backbone of ResNet-18	0.95
YOLO v2 with ADAM optimizer	0.96
YOLO v2 without data augmentation	0.88
YOLO v2 with SGD optimizer	0.97
YOLO v3 with SGD optimizer	0.97

Figure 3(b) and (c) shows the object tracking using the Kalman filter. Table 2 shows the results of the pear counting. The counting results obtained by the present method, method without data

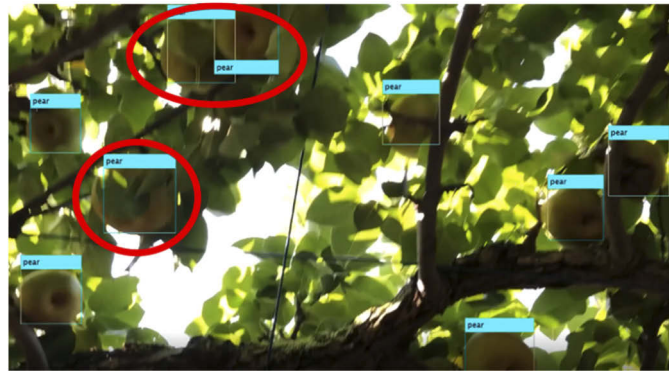


Fig. 4. An enlarged view of the video frame after the pear detection. The detected pears were represented by the rectangles. As the red circle indicates, the pear partly covered by leaf and overlapped pear could be detected.

augmentation, and YOLO v2 network with the backbone network of ResNet-18 with an input size of 224×224 are shown. A total of 234 pears were included in the video for the test data. For the present method, the number of detected pears was 231. For the evaluation, we have to consider the omission, double count, and wrong detection. When an object other than pear was detected as a pear, it was regarded as a wrong detection. Considering them, the number of correctly counted pears was 226. The F1 value was 0.972. The numbers of correctly counted pears were 196 and 211, without data augmentation and at an input size of 224×224 , respectively. The number of pears obtained with the KLT tracker in lieu of the Kalman filter was 310, which implies that the pears were over-counted. The precision, recall, and F1 value in the apple counting were 0.935, 0.924, and 0.929, respectively. The number of correctly detected apples was 157, out of 170 apples. The apple counting was also carried out accurately. The detail is shown in Table 3.

Table 2. Result of the pear counting.

Method	Detected number	Omission	Double count	Wrong detection	Correct detection	Number	Precision	Recall	F1
Present method	231	8	4	1	226	234	0.978	0.966	0.972
No data augmentation	206	38	8	2	196	234	0.951	0.838	0.891
With smaller input image size	234	23	2	21	211	234	0.902	0.902	0.902

Table 3. Result of the apple counting.

Method	Detected number	Omission	Double count	Wrong detection	Correct detection	Number	Precision	Recall	F1
Present method	168	13	3	8	157	170	0.935	0.924	0.929

Figure 5 illustrates the Kalman filter operation. Figure 5(a) does not contain a bounding box, which implies that the pear at the centre of the panel was not detected, while the pear could be detected in the previous and subsequent frames. For clarification, a dotted circle is presented around the pear that was not detected. As shown in Fig. 5(b), the pear could be tracked because the pear was tracked by the Kalman filter from the previous frames. This shows that the pear could be tracked even when it was not detected in some frames in the image sequences. When the tracking was not satisfactory, the pear was counted twice, which led to over-counting. On the

other hand, a pear could not be detected, as shown in Fig. 5(c) as the light conditions suddenly changed. This pear could be detected in the previous and subsequent frames. The pear that should have been detected is outlined by a dotted circle. As shown in Fig. 5(d), this pear could be tracked similarly using the information in the frames.

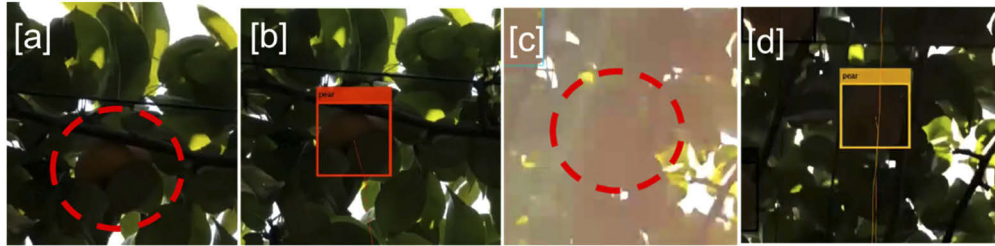


Fig. 5. The detection result with YOLO detector and the object tracking with Kalman filter. The panel [a] and [c] depict the result of the detection using YOLO v2 network. The pears rounded by dotted circles were not detected due to the illumination condition. On the other hand, the pears in the panel [b] and [d] could be tracked using Kalman filter.

4. Discussion

Both pear detection and counting could be accurately performed. The absolute error of the pear counting was smaller than 10%, owing to the 1) high average precision for pear detection and 2) good performance of the fruit tracking using the Kalman filter. As indicated by the red-circled pears in Fig. 4, the pears that were overlapped and partly covered by leaves could be successfully detected. The detection of overlapped fruits with a green background by a classical image processing method is challenging. When the background was excluded using the green colour threshold, the pears were also excluded. The input size for the YOLO network was 608×608 pixels, which also contributed to the accurate fruit detection. As shown in Table 1, when the ResNet-18-based network was used and the input size was 224×224 , the average precision was 0.95. The input image was convoluted after the image was fed into the network. The feature map was then calculated. With the increase in the size (i.e., height and width) of the feature map, smaller objects tended to be detected accurately. This can explain the higher performance of the fruit detection when the input size of the image was larger. Data augmentation was employed to alter the colour information during the training. The average precision with the data augmentation was 0.97, significantly higher than that without data augmentation (0.88). The colour of fruits under the tree crown, particularly under the canopy, is susceptible to change by the light conditions and measurement time. Thus, the data augmentation contributed to the high average precision. The apple detection performance was also good. A green apple was also detected. This may provide yield estimation and detection systems for greenish cultivars.

The accurate object tracking with the Kalman filter enabled an accurate fruit counting. As many object detectors have been developed, some strategies are available for the fruit detection. However, fruit counting is still challenging since to associate the detected fruits with new detection result is difficult especially when the illumination condition or recording condition of videos are unstable. This study not only detected fruits accurately by using YOLO detector, but also tracked the detected fruits for the accurate fruit counting under poor illumination condition. Using the Kalman filter, the positions of the fruits in the frame that is one frame ahead can be estimated. Even if the fruits cannot be found in the next frame, the object can be tracked as long as it can be detected later. As shown in Fig. 5, even when the pear was not detected owing to changes in the surrounding physical environment or illumination conditions, the pear could be tracked. Because the light conditions are prone to change while recording the video, the Kalman filter is

very effective for the counting of fruits. The position of the detected fruit in the next frame was predicted based on the information after the fruit was detected for the first time. This study is significant in the terms of the accurate detection and tracking of the detected fruits. However, if a fruit moves outside the frame and the fruit is recorded later, it becomes difficult to identify it, resulting in a double count. A blurred image can lead to a difficulty in counting. If the image is recorded by robots moving on the ground, the image can be blurred if the ground is bumpy. Those can be a limitation and concern in this method. When the KLT tracker was used for pear tracking, the pears were over-counted, because it was challenging to track an identical pear when the pear was not detected in the subsequent frame. Many efforts have been made for the object detector. We would like to test some more deep learning-based detectors like EfficientDet [18] and YOLO v4 [19]. By the KLT tracker, the fruits were tracked using image features. The tracking with the Kalman filter was more accurate because the position of the next frame was predicted. This fruit counting method could be applied in various fields such as livestock management, phenotyping, agricultural machinery, and evaluation of natural conservation. In our future work, it is preferable to conduct more experiments in other fields and to confirm if our method is significantly higher than other compared methods. In the application of this method, the video acquisition can be automated such as by drones and robots. In that case, the region of interest should be defined by us. In our study, more trees existed in the experimental field. The area to be used for the fruit counting is selected by the user. A manual process would be included there. A method that automatically crop the videos for the counting is desired.

Funding. Japan Society for the Promotion of Science (JP19J22701).

Acknowledgments. We are very grateful to Hatsuseien and Fruit Production, which offered the test field for our study.

Disclosures. The authors declare no conflicts of interest.

Data availability. Data underlying the results presented in this paper are not available. The data includes the secret skills in making a high-quality fruit of the farmers.

References

1. K. Yamamoto, W. Guo, Y. Yoshioka, and S. Ninomiya, "On plant detection of intact tomato fruits using image analysis and machine learning methods," *Sensors* **14**(7), 12191–12206 (2014).
2. S. Takemoto, Y. Harada, and K. Imai, "Image-based Determination of Plum "Tsuyuakane" Ripeness via Deep Learning," *Agri. Info. Res.* **28**(3), 108–114 (2019).
3. Y. Song, C. A. Glasbey, G. W. Horgan, G. Polder, J. A. Dieleman, and G. W. A. M. van der Heijden, "Automatic fruit recognition and counting from multiple images," *Biosyst. Eng.* **118**, 203–215 (2014).
4. T. T. Santos, L. L. de Souza, A. A. dos Santos, and S. Avila, "Grape detection, segmentation, and tracking using deep neural networks and three-dimensional association," *Comput. Electron. Agric.* **170**, 105247 (2020).
5. K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision* (2017), pp. 2961–2969.
6. H. Kang and C. Chen, "Fruit detection, segmentation and 3D visualisation of environments in apple orchards," *Comput. Electron. Agric.* **171**, 105302 (2020).
7. S. Wan and S. Goudos, "Faster R-CNN for multi-class fruit detection using a robotic vision system," *Comput. Networks* **168**, 107036 (2020).
8. J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2017), pp. 7263–7271.
9. J. Redmon and A. Farhadi, *YOLOv3: An Incremental Improvement* (2018).
10. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *European Conference on Computer Vision* (2016), pp. 21–37.
11. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (IEEE Computer Society, 2016), Vol. 2016-December, pp. 770–778.
12. I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep learning*. MIT press, Cambridge (2016).
13. P. Henderson and V. Ferrari, "End-to-end training of object class detectors for mean average precision," in *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* (Springer Verlag, 2017), Vol. 10115 LNCS, pp. 198–213.
14. G. Welch and G. Bishop, "An introduction to the Kalman filter". TR 95–041. University of North Carolina at Chapel Hill, Department of Computer Science (1995).

15. X. Li, K. Wang, W. Wang, and Y. Li, "A multiple object tracking method using Kalman filter," in *2010 IEEE International Conference on Information and Automation, ICIA 2010* (2010), pp. 1862–1866.
16. B. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proceedings DARPA Image Understanding Workshop* (1981), pp. 121–130.
17. J. Shi and C. Tomasi, "Good features to track," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (Publ by IEEE, 1994), pp. 593–600.
18. M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (2019), 10778–10787.
19. A. Bochkovskiy, C. Y. Wang, and H. Y. M Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. *ArXiv*. (2020)